> **NOTE** Setting up the Google provider requires an API key, which can be obtained by having a valid Google project and generating one in the AI studio page at https://aistudio.google.com/app/apikey.

In this implementation, we define an object called `supportedProviders` that maps provider names to their respective configurations. Each configuration includes the provider constructor function and a list of supported models for that provider.

The `getSupportedModel` function takes two arguments: provider (the provider name) and model (the desired model name). After that, it checks its parameters are correct and the API key is present in the environment. It then calls the provider constructor function with the API key to create an instance of the provider and returns the requested model instance by calling the provider instance with the model's name.

> **NOTE** While the current implementation defines supported providers directly in the code, a more flexible and maintainable approach would be to use external configuration files (like YAML or JSON). Another option is to create a dedicated configuration page where users can enter provider credentials and preferences, which are then stored securely in a database. Both methods make it easier to add or modify AI providers without changing the core application code.

The returned model should be usable within the `streamText` function, and I show how to use this in listing 3.12. For clarity, I show only the half of the code that includes the `getSupportedModel`, as the rest of the code remains the same.

Listing 3.12 Using `getSupportedModel` in the route handler

```
import { StreamingTextResponse, streamText, StreamData } from 'ai';
import { getSupportedModel } from './utils';

export const dynamic = 'force-dynamic';

export async function POST(req) {
  const { messages, provider, model } = await req.json();
  const result = await streamText({
    model: getSupportedModel(provider, model);
    maxTokens: 512,
   ...
```

Imports the getSupportedModel function

Extracts the provider and model preference from the request object

Gets the supported model instance using the getSupportedModel function

By using the `getSupportedModel` function, we can dynamically select the appropriate model instance based on the provider and model values provided in the request body. This approach allows for flexibility and scalability, making it easy to add more providers in the future.

Now the client-side application needs to send the provider and model values in the request body, and the API route handler will automatically select the corresponding model instance. We will see how to update the `useChat` hook to include those parameters next.